

คู่มือฉบับมือใหม่ 100%

เอกสารนี้อธิบาย "หลักการเบื้องหลัง" ของโค้ดทั้งหมดในไฟล์ **คู่มือสร้างเว็บเร่งด่วน.pdf** แบบละเอียดที่สุด เหมาะสำหรับ คนที่ไม่เคยเขียน PHP มาก่อนเลย อ่านไฟล์นี้ก่อน แล้วค่อยกลับไปดูโค้ดในไฟล์คู่มือสร้างเว็บเร่งด่วน จะเข้าใจว่าทำไมต้องเขียนแบบนั้น

สารบัญ

1. PHP คืออะไร ทำงานอย่างไร
2. PHP ผสมกับ HTML ได้อย่างไร
3. ตัวแปรและชนิดข้อมูลพื้นฐาน
4. โครงสร้างควบคุม: if, for, while, foreach
5. ฟังก์ชันคืออะไร
6. ฟอรั่ม HTML ส่งข้อมูลไปหา PHP ได้อย่างไร (\$_POST, \$_GET)
7. ฐานข้อมูล MySQL คืออะไร เชื่อมต่ออย่างไร (mysqli)
8. Prepared Statement คืออะไร ทำไมต้องใช้
9. AJAX และ jQuery คืออะไร ทำไมหน้าเว็บไม่ reload
10. ไฟล์โค้ดที่ละบรรทัด: connect.php
11. ไฟล์โค้ดที่ละบรรทัด: index.php
12. ไฟล์โค้ดที่ละบรรทัด: search.php + AJAX ค้นหา
13. ไฟล์โค้ดที่ละบรรทัด: delete.php + AJAX ลบ
14. ไฟล์โค้ดที่ละบรรทัด: get_one.php / update.php + AJAX แก้ไข
15. Error ที่เจอบ่อยตอนเริ่มต้น และวิธีแก้
16. คำศัพท์ที่ควรรู้ (Glossary)

1. PHP คืออะไร ทำงานอย่างไร

PHP เป็นภาษาที่ทำงาน **ฝั่งเซิร์ฟเวอร์ (server-side)** ซึ่งต่างจาก HTML และ JavaScript ที่ทำงาน **ฝั่งผู้ใช้ (client-side)**

เปรียบเทียบง่ายๆ: ลองนึกภาพร้านอาหาร — คุณ (browser) สั่งอาหารกับพนักงาน (ส่ง request ไปที่เซิร์ฟเวอร์) พนักงานเดินเข้าครัว (เซิร์ฟเวอร์รัน PHP) ปิ้งอาหารตามสูตร อาจต้องไปหยิบวัตถุดิบจากตู้เย็น (ดึงข้อมูลจากฐานข้อมูล MySQL) แล้วนำอาหารที่ปิ้งเสร็จ (HTML ที่สร้างเสร็จแล้ว) มาเสิร์ฟให้คุณ — คุณจะไม่เห็นขั้นตอนการปิ้งในครัวเลย เห็นแค่ผลลัพธ์สุดท้ายที่เป็นจานอาหาร (หน้าเว็บ HTML)

ขั้นตอนจริงเมื่อเปิดเว็บที่ใช้ PHP:

1. ผู้ใช้พิมพ์ URL หรือกดปุ่มในหน้าเว็บ (browser ส่ง request ไปหาเซิร์ฟเวอร์ เช่น Apache ใน XAMPP)
2. เซิร์ฟเวอร์เห็นว่าไฟล์ที่ขอมามีนามสกุล `.php` จึงส่งให้โปรแกรม PHP รันโค้ดนั้น
3. PHP อาจไปคุยกับฐานข้อมูล MySQL เพื่อดึงหรือบันทึกข้อมูล
4. PHP สร้างผลลัพธ์เป็นโค้ด HTML ธรรมดา แล้วส่งกลับไปให้ browser

5. Browser แสดงผล HTML ที่ได้รับ — ผู้ใช้จะไม่เห็นโค้ด PHP เลยแม้แต่บรรทัดเดียว (ถ้ากด "View Source" จะเห็นแต่ HTML ล้วนๆ)

นี่คือเหตุผลที่ไฟล์ index.php ในคู่มือเล่มก่อนหน้ามีทั้งโค้ด PHP (ใน `<?php ... ?>`) และโค้ด HTML ปั่นกันอยู่ในไฟล์เดียวได้ — เซิร์ฟเวอร์จะรันส่วน PHP ก่อน แล้วค่อยส่ง HTML ที่ได้ (รวมถึงส่วน HTML ที่เขียนไว้ปกติ) ออกไปให้ผู้ใช้ทีเดียว

2. PHP ผสมกับ HTML ได้อย่างไร

ไฟล์ PHP คือไฟล์ HTML ธรรมดาที่แทรกโค้ด PHP เข้าไปตรงไหนก็ได้ โดยเปิด-ปิดด้วยสัญลักษณ์พิเศษ:

```
<p>ข้อความ HTML ปกติ</p>

<?php
    echo "ข้อความนี้มาจาก PHP";
?>

<p>ข้อความ HTML ปกติต่อ</p>
```

ทุกครั้งที่เราเขียน **<?php** คือการ "สลับโหมด" จาก HTML ไปเป็น PHP ให้เซิร์ฟเวอร์เริ่มประมวลผลโค้ดที่ตามมา และ **?>** คือการสลับกลับไปเป็น HTML ธรรมดา คำสั่ง `echo` คือคำสั่งที่บอกให้ PHP "พิมพ์ข้อความนี้ออกไปเป็น HTML" — เกือบทุกครั้งที่ต้องการให้ค่าตัวแปรไปโผล่บนหน้าเว็บ ต้องใช้ echo (หรือทางลัด `<?= ... ?>` ซึ่งเหมือนกับ `<?php echo ... ?>`)

ตัวอย่างทางลัดที่ใช้บ่อยมากในไฟล์ index.php:

```
<td><?= htmlspecialchars($row['name']) ?></td>
```

อ่านว่า "แสดงค่าของ `$row['name']` (ผ่านการทำความสะอาดด้วย htmlspecialchars ก่อน) ตรงตำแหน่งนี้ในตาราง HTML" เครื่องหมาย `<?=` คือทางลัดของ `<?php echo` นิยมใช้เวลาต้องการแทรกค่าตัวแปรลงกลาง HTML สั้นๆ แบบนี้

3. ตัวแปรและชนิดข้อมูลพื้นฐาน

ตัวแปรใน PHP เริ่มต้นด้วย `$` เสมอ และไม่ต้องประกาศชนิดข้อมูลล่วงหน้า (PHP เดาชนิดข้อมูลให้อัตโนมัติ)

```
$name = "สมชาย";      // ชนิด string (ข้อความ) ต้องอยู่ในเครื่องหมายคำพูด
$age  = 15;           // ชนิด int (จำนวนเต็ม)
$price = 29.50;       // ชนิด float (ทศนิยม)
$isActive = true;     // ชนิด boolean (true/false)
$colors = ["red", "green", "blue"]; // ชนิด array (ชุดข้อมูล)
```

Array คือตัวแปรที่เก็บข้อมูลได้หลายค่าในตัวเดียว มี 2 แบบที่พบบ่อย:

- **Indexed array:** ใช้ตัวเลข (0,1,2,...) อ้างอิงอัตโนมัติ เช่น `$colors[0]` จะได้ "red"
- **Associative array:** ผู้เขียนตั้งชื่อ key เอง เช่น `$student = ["name" => "สมชาย", "age" => 15];` เข้าถึงด้วย `$student['name']`

ในโค้ดคู่มือก่อนหน้านี้ ตัวแปร `$row` ที่ได้จากฐานข้อมูล (`$row = $result->fetch_assoc();`) คือ Associative array — แต่ละแถวข้อมูลจากฐานข้อมูลจะถูกแปลงมาเป็น array ที่ key ตรงกับชื่อคอลัมน์ เช่น `$row['name']`, `$row['age']`

4. โครงสร้างควบคุม: if, for, while, foreach

if — การตัดสินใจ

```
if ($age >= 18) {  
    echo "เป็นผู้ใหญ่";  
} else {  
    echo "ยังไม่บรรลุนิติภาวะ";  
}
```

อ่านว่า "ถ้า \$age มากกว่าหรือเท่ากับ 18 ให้พิมพ์ว่าเป็นผู้ใหญ่ ไม่เช่นนั้นให้พิมพ์ว่ายังไม่บรรลุนิติภาวะ" — เครื่องหมายที่ใช้เทียบค่าที่ต้องรู้: `==` (เท่ากับ), `!=` (ไม่เท่ากับ), `>` `<` `>=` `<=`

while — วงซ้ำจนกว่าเงื่อนไขจะเป็นเท็จ

```
while ($row = $result->fetch_assoc()) {  
    echo $row['name'];  
}
```

รูปแบบนี้พบบ่อยมากตอนดึงข้อมูลจากฐานข้อมูล — `fetch_assoc()` จะดึงข้อมูลมาทีละ 1 แถว ทุกครั้งที่เรียก แล้วขยับไปแถวถัดไปเองอัตโนมัติ เมื่อไม่มีแถวเหลือแล้วจะคืนค่า `false` ทำให้เงื่อนไขของ while เป็นเท็จและหยุดวน — จึงใช้ while วนอ่านข้อมูล "จนกว่าจะหมดตาราง" โดยไม่ต้องรู้ล่วงหน้าว่ามีกี่แถว

for — วงซ้ำตามจำนวนรอบที่กำหนด

```
for ($i = 1; $i <= 5; $i++) {  
    echo $i;  
}
```

โครงสร้างคือ (ค่าเริ่มต้น; เงื่อนไขที่ต้องจริงถึงจะวนต่อ; การเปลี่ยนค่าทุกจบรอบ) ตัวอย่างนี้จะพิมพ์ 1 2 3 4 5

foreach — วนอ่านค่าทุกตัวใน array

```
foreach ($colors as $c) {  
    echo $c;  
}
```

ใช้เมื่อมี array อยู่แล้วและต้องการวนอ่านค่าทุกตัวในนั้นทีละตัว โดยไม่ต้องนับ index เอง

5. ฟังก์ชันคืออะไร

ฟังก์ชันคือ "ชุดคำสั่งที่ตั้งชื่อไว้ใช้ซ้ำได้" แทนที่จะเขียนโค้ดเดิมซ้ำหลายที่ ให้ห่อไว้ในฟังก์ชันแล้วเรียกใช้ชื่อแทน

```
function sumTwo($a, $b) {  
    return $a + $b;  
}  
  
echo sumTwo(3, 5); // ได้ 8
```

`$a` และ `$b` คือ "พารามิเตอร์" (ตัวแปรรับค่าที่ส่งเข้ามาตอนเรียกใช้) `return` คือคำสั่งส่งค่าผลลัพธ์ออกจากฟังก์ชันกลับไปให้จุดที่เรียกใช้ ฟังก์ชันที่ใช้อยู่ในคู่มือนี้ล้วนเป็นฟังก์ชันที่ PHP เตรียมไว้ให้แล้ว (built-in function) เช่น `htmlspecialchars()`, `mysqli_connect()`, `json_encode()` — ไม่ต้องเขียนเองตั้งแต่ต้น แค่เรียกใช้ตามชื่อ

6. ฟอรัม HTML ส่งข้อมูลไปหา PHP ได้อย่างไร

```
<form method="post">
  <input type="text" name="name">
  <button type="submit">บันทึก</button>
</form>
```

เมื่อผู้ใช้พิมพ์ข้อความในช่อง input แล้วกดปุ่ม submit เกิดอะไรขึ้น:

- 1) Browser รวบรวมค่าจาก input ทุกตัวที่มี attribute `name`
 - 2) เพราะ `method="post"` จึงส่งค่าเหล่านั้นซ่อนไปใน "เนื้อหา" ของ request (ไม่โผล่ใน URL)
 - 3) ฟัง PHP รับค่าผ่านตัวแปรพิเศษชื่อ `$_POST` ซึ่งเป็น associative array ที่ key ตรงกับ attribute `name` ของ input นั้นๆ เช่น `$_POST['name']`
- ถ้าใช้ `method="get"` แทน ค่าจะถูกส่งไปใน URL และฝั่ง PHP ต้องใช้ `$_GET['name']` แทน

เปรียบเทียบ: `$_POST` เหมือนซองจดหมายที่ปิดผนึก (ข้อมูลอยู่ข้างในมองจากภายนอกไม่เห็น) ส่วน `$_GET` เหมือนโปสการ์ดที่ข้อความเขียนโชว์อยู่ด้านหลังบัตร (ใครก็เห็นได้ทันที เพราะมันอยู่ใน URL)

7. ฐานข้อมูล MySQL คืออะไร เชื่อมต่ออย่างไร

MySQL คือโปรแกรมสำหรับเก็บข้อมูลอย่างเป็นระบบในรูปแบบ "ตาราง" (คล้าย Excel ที่มีแถวและคอลัมน์) PHP เองไม่ได้เก็บข้อมูลถาวร — ทุกครั้งที่ปิด browser ตัวแปรใน PHP จะหายไปหมด จึงต้องพึ่งฐานข้อมูลเพื่อเก็บข้อมูลไว้ใช้ในครั้งต่อไป

```
$conn = mysqli_connect("localhost", "root", "", "school_db");
```

`mysqli_connect()` คือฟังก์ชันที่ใช้ "เปิดสาย" เชื่อมต่อไปยังโปรแกรม MySQL พารามิเตอร์ 4 ตัวคือ

- 1) **"localhost"** = ที่อยู่ของเครื่องที่รัน MySQL (เครื่องเดียวกับที่รัน PHP ผ่าน XAMPP)
- 2) **"root"** = ชื่อผู้ใช้ที่จะ login เข้า MySQL (ค่าเริ่มต้นของ XAMPP)
- 3) **""** = รหัสผ่าน (XAMPP ค่าเริ่มต้นไม่ตั้งรหัสผ่าน จึงเว้นว่างไว้)
- 4) **"school_db"** = ชื่อฐานข้อมูลที่ต้องการใช้งาน

ค่าที่คืนกลับมาเก็บไว้ในตัวแปร `$conn` ซึ่งจะถูกใช้ต่อทุกครั้งที่ต้องการสั่งงานฐานข้อมูล (เช่น `$conn->query(...)`, `$conn->prepare(...)`)

8. Prepared Statement คืออะไร ทำไมต้องใช้

วิธีที่ "ดูเหมือนง่าย" แต่ **อันตรายมาก** คือการต่อ string ตรงๆ แบบนี้ (ห้ามเขียนแบบนี้):

```
// อย่าเขียนแบบนี้ !
$name = $_POST['name'];
$conn->query("SELECT * FROM students WHERE name = '$name'");
```

อันตราย: ถ้าผู้ใช้พิมพ์ข้อความแปลกๆ ลงในช่อง name เช่น `' OR '1'='1` คำสั่ง SQL ที่แท้จริงจะกลายเป็น `WHERE name = '' OR '1'='1'` ซึ่งเงื่อนไข `'1'='1'` เป็นจริงเสมอ ทำให้ดึงข้อมูลทุกแถวออกมาซึ่งไม่ควรจะเห็น นี่คือการโจมตีที่เรียกว่า **SQL Injection**

วิธีที่ปลอดภัยคือใช้ **Prepared Statement**:

```
$stmt = $conn->prepare("SELECT * FROM students WHERE name = ?");
$stmt->bind_param("s", $name);
$stmt->execute();
$result = $stmt->get_result();
```

อธิบายทีละบรรทัด:

- 1) `prepare(...)`: ส่งแค่ "โครงสร้าง SQL" ไปให้ MySQL เตรียมไว้ก่อน โดยใช้เครื่องหมาย `?` แทนตำแหน่งที่จะใส่ค่าจริงทีหลัง (MySQL จะรู้แน่นอนว่าตรงนี้เป็น "ค่าข้อมูล" ไม่ใช่ส่วนหนึ่งของคำสั่ง)
- 2) `bind_param("s", $name)`: นำค่าจริง (\$name) มาผูกเข้ากับเครื่องหมาย `?` ตัวอักษร `"s"` บอกชนิดข้อมูล (s=string, i=integer, d=double/float) ถ้ามีหลาย `?` ต้องระบุตัวอักษรและตัวแปรให้ครบตามลำดับ เช่น `bind_param("sis", $name, $age, $class)`
- 3) `execute()`: สั่งให้รันคำสั่งจริง
- 4) `get_result()`: ดึงผลลัพธ์ที่ได้ออกมาเพื่อนำไปอ่านต่อด้วย `fetch_assoc()`

ข้อดีคือไม่ว่าผู้ใช้จะพิมพ์อะไรมาก็ตาม คำนั้นจะถูกมองเป็น "ข้อมูล" เท่านั้น ไม่มีทางกลายเป็นคำสั่ง SQL ปลอมได้เลย

9. AJAX และ jQuery คืออะไร ทำไมหน้าเว็บไม่ reload

ปกติเมื่อกดปุ่ม submit ฟอรัม browser จะโหลดหน้าเว็บใหม่ทั้งหน้า (กระพริบขาววับ) แต่เว็บสมัยใหม่ เช่น การค้นหาที่พิมพ์แล้วผลลัพธ์เปลี่ยนทันที ไม่ต้องรอโหลดหน้าใหม่ เบื้องหลังคือเทคนิคที่เรียกว่า **AJAX**

เปรียบเทียบ: ถ้าไม่มี AJAX การขอข้อมูลใหม่เหมือนต้องเดินออกจากบ้านไปใหม่ทั้งหมดแล้วเข้ามาใหม่ (reload ทั้งหน้า) ส่วน AJAX เหมือนแค่ยกมือเรียกพนักงานมาที่โต๊ะโดยไม่ต้องลุกออกจากบ้าน (ส่ง request เล็กๆ ไปเซิร์ฟเวอร์เบื้องหลัง แล้วอัปเดตแค่บางส่วนของหน้าเว็บ)

jQuery คือไลบรารี (ชุดคำสั่งสำเร็จรูป) ของ JavaScript ที่ทำให้เขียนโค้ดสั่งงาน AJAX และจัดการหน้าเว็บได้ง่ายและสั้นลงมาก

```
$.ajax({
  url: 'search.php',
  method: 'POST',
  data: { keyword: $('#searchBox').val() },
  success: function(response){
    $('#studentTable').html(response);
  }
});
```

อธิบายทีละส่วน:

- `url: 'search.php'` คือไฟล์ PHP ปลายทางที่จะรับข้อมูลไปประมวลผล (ทำงานเบื้องหลัง ไม่มีการเปลี่ยนหน้าให้เห็น)
 - `method: 'POST'` คือวิธีส่งข้อมูล เหมือนกับตอนส่งฟอร์ม
 - `data: { keyword: ... }` คือข้อมูลที่จะส่งไป ซึ่งฝั่ง search.php จะรับผ่าน `$_POST['keyword']`
 - `success: function(response){ ... }` คือคำสั่งที่จะทำงาน "เมื่อได้รับคำตอบกลับมาจาก search.php สำเร็จ" ตัวแปร `response` คือข้อความ/HTML ที่ search.php echo กลับมา
 - `$('#studentTable').html(response)` คือคำสั่ง jQuery ที่บอกว่า "นำ HTML ใน response ไปแทนที่เนื้อหาภายใน element ที่มี id เท่ากับ studentTable" — ตรงนี้เองที่ทำให้ตารางเปลี่ยนแปลงโดยไม่ต้อง reload หน้า
- สัญลักษณ์ `$('#xxx')` คือวิธีของ jQuery ในการ "เลือก" element ตาม id (ใช้ # นำหน้าเหมือนใน CSS) ส่วน `$('.xxx')` ใช้เลือกตาม class

10. โค้ดที่ละบรรทัด: connect.php

```
<?php
$conn = mysqli_connect("localhost", "root", "", "school_db");
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
?>
```

บรรทัดที่ 1 **<?php**: เริ่มโหมด PHP

บรรทัดที่ 2: เปิดการเชื่อมต่อฐานข้อมูล เก็บผลลัพธ์ไว้ในตัวแปร \$conn (ถ้าสำเร็จจะได้ connection object ถ้าไม่สำเร็จจะได้ค่า false)

บรรทัดที่ 3-5: **if (!\$conn)** อ่านว่า "ถ้า \$conn ไม่เป็นความจริง" (เครื่องหมาย **!** คือ "ไม่") ซึ่งจะเกิดขึ้นเมื่อเชื่อมต่อไม่สำเร็จ (เช่น XAMPP ยังไม่เปิด MySQL หรือพิมพ์ชื่อฐานข้อมูลผิด) — **die()** คือคำสั่งหยุดการทำงานของสคริปต์ทั้งหมดทันทีพร้อมแสดงข้อความ error ที่ระบุไว้ เพื่อให้รู้ทันทีว่าปัญหาอยู่ที่การเชื่อมต่อ ไม่ใช่ปล่อยให้โค้ดทำงานต่อแล้ว error แบบงงๆ ที่หลัง
บรรทัดสุดท้าย **?>**: จบโหมด PHP

11. โค้ดที่ละบรรทัด: index.php

```
<?php
require "connect.php";

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $name = $_POST['name'];
    $age = $_POST['age'];
    $class = $_POST['class'];

    $stmt = $conn->prepare("INSERT INTO students (name, age, class) VALUES (?, ?, ?)");
    $stmt->bind_param("sis", $name, $age, $class);
    $stmt->execute();
}

$result = $conn->query("SELECT * FROM students ORDER BY id DESC");
?>
```

require "connect.php"; — ดึงโค้ดจากไฟล์ connect.php มารวมกับไฟล์นี้ ทำให้ได้ตัวแปร \$conn มาใช้ต่อโดยไม่ต้องเขียนคำสั่งเชื่อมต่อซ้ำ

if (\$_SERVER['REQUEST_METHOD'] == 'POST') — ไฟล์ index.php ไฟล์เดียวนี้ถูกเปิดได้ 2 สถานการณ์: (ก) เปิดครั้งแรกแบบปกติ (เป็น GET) กับ (ข) เปิดหลังจากผู้ใช้กดปุ่ม submit ฟอรัมที่อยู่ในหน้าเดียวกัน (เป็น POST) เงื่อนไขนี้เช็คค่า "รอบนี้ผู้ใช้กด submit มาหรือเปล่า" ถ้าใช่ค่อยรันโค้ดบันทึกข้อมูล ถ้าไม่ใช่ (เปิดหน้าเฉยๆ) ก็ข้ามส่วนนี้ไปเลย

\$name = \$_POST['name']; ... — ดึงค่าจากฟอร์มมาเก็บในตัวแปรที่ชื่อสั้นและเข้าใจง่ายกว่า

\$stmt = \$conn->prepare(...) ถึง **execute()** — ใช้ Prepared Statement บันทึกข้อมูลใหม่ลงตาราง students ตามที่อธิบายไว้ในหัวข้อที่ 8

\$result = \$conn->query("SELECT * FROM students ORDER BY id DESC"); — ดึงข้อมูลทุกแถวจากตาราง students มาเก็บไว้ในตัวแปร \$result เรียงจาก id มากไปน้อย (คนที่เพิ่มล่าสุดจะอยู่บนสุด) หมายเหตุ: บรรทัดนี้ใช้ `query()` ตรงๆ ได้เพราะไม่มีค่าจากผู้ใช้มาปนในคำสั่ง จึงไม่มีความเสี่ยง SQL Injection (ต่างจากตอน INSERT ที่มีค่าจากฟอร์มมาปนจึงต้องใช้ prepared statement)

```
<?php while ($row = $result->fetch_assoc()) { ?>
<tr>
    <td><?= htmlspecialchars($row['name']) ?></td>
    <td><?= htmlspecialchars($row['age']) ?></td>
    <td><?= htmlspecialchars($row['class']) ?></td>
</tr>
<?php } ?>
```

ส่วนนี้คือจุดที่ PHP กับ HTML ผสมกันชัดเจนที่สุด: `<?php while (...) { ?>` เปิดโหมด PHP เพื่อเริ่มวนลูป แล้ว **ปิดโหมด PHP ทันทีด้วย** `?>` เพื่อเขียน HTML ของ 1 แถวตารางตามปกติ (โดยแทรกค่าตัวแปรด้วย `<?= ?>`) จากนั้นเปิดโหมด PHP อีกครั้งด้วย `<?php } ?>` เพื่อปิดวงเล็บของ while — ทุกครั้งที่วนลูปหนึ่งรอบ (หนึ่งแถวข้อมูล) โค้ด HTML ตรงกลางจะถูกพิมพ์ออก

มาซ้ำหนึ่งชุด ทำให้ได้ตารางที่มีจำนวนแถวเท่ากับจำนวนข้อมูลในฐานข้อมูลพอดี

htmlspecialchars() ห่อไว้ทุกครั้งที่แสดงค่าจากฐานข้อมูล เพื่อป้องกันว่าถ้ามีใครเคยกรอกอักขระ HTML แปลกๆ ไว้ในชื่อ (เช่น `<script>`) มันจะไม่ถูกตีความเป็นโค้ดจริงตอนแสดงผล

12. ไลค์โค้ดที่ละบรรทัด: search.php + AJAX ค้นหา

```
<?php
require "connect.php";
$keyword = $_POST['keyword'];

$stmt = $conn->prepare("SELECT * FROM students WHERE name LIKE ?");
$search = "%$keyword%";
$stmt->bind_param("s", $search);
$stmt->execute();
$result = $stmt->get_result();
?>
```

\$keyword = \$_POST['keyword']; — รับค่าค้นที่ jQuery ส่งมาจากช่อง input

LIKE ? — คำสั่ง SQL **LIKE** ใช้ค้นหาแบบ "มีคำนี้อยู่บางส่วน" ต่างจาก **=** ที่ต้องตรงกันเป๊ะทั้งหมด

\$search = "%\$keyword%"; — เครื่องหมาย **%** ใน SQL แปลว่า "ข้อความอะไรก็ได้" ดังนั้น **"%สม%"** จะจับคู่กับ "สมชาย", "สมหญิง", "อารีสม" ได้หมด เพราะมีคำว่า "สม" อยู่ตรงไหนก็ได้ในชื่อ
ที่เหลือเหมือนหัวข้อที่ 8 ทุกประการ (prepare → bind_param → execute → get_result)

เมื่อผู้ใช้พิมพ์ในช่องค้นหา jQuery ฝั่ง index.php จะทำงานดังนี้:

```
$('#searchBox').on('keyup', function(){
    $.ajax({
        url: 'search.php',
        method: 'POST',
        data: { keyword: $(this).val() },
        success: function(response){
            $('#studentTable').html(response);
        }
    });
});
```

\$('#searchBox').on('keyup', function(){...}) — ผูก "เหตุการณ์" (event) ไว้กับช่อง input ที่มี id="searchBox" ว่า "ทุกครั้งที่มีการกดปล่อยแป้นพิมพ์ (keyup) ในช่องนี้ ให้รันฟังก์ชันข้างในนี้"

\$(this).val() — **this** หมายถึง element ที่เหตุการณ์เกิดขึ้น (คือช่อง input ที่กำลังพิมพ์อยู่) **.val()** คือคำสั่งดึง "ค่าปัจจุบัน" ที่พิมพ์อยู่ในช่องนั้นออกมา

ที่เหลือคือการยิง AJAX ไปหา search.php ตามที่อธิบายในหัวข้อที่ 9

13. โล้โค้ดที่ลบบรรทัด: delete.php + AJAX ลบ

```
<?php
require "connect.php";
$id = $_POST['id'];

$stmt = $conn->prepare("DELETE FROM students WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();

echo "success";
?>
```

รับ `id` ของแถวที่ต้องการลบ แล้วรันคำสั่ง DELETE ด้วย Prepared Statement เช่นเดิม บรรทัดสุดท้าย `echo "success";` คือการ "ตอบกลับ" ไปบอกฝั่ง jQuery ว่าลบสำเร็จแล้ว ข้อความนี้จะไปโผล่เป็นค่าตัวแปร `response` ในฝั่ง jQuery

```
$(document).on('click', '.btn-delete', function(){
    if (confirm('ยืนยันการลบข้อมูลนี้หรือไม่?')) {
        let id = $(this).data('id');
        let btn = $(this);
        $.post('delete.php', { id: id }, function(response){
            if (response.trim() === 'success') {
                btn.closest('tr').remove();
            }
        });
    }
});
```

\$(document).on('click', '.btn-delete', ...) — วิธีนี้เรียกว่า "event delegation" ผูก event ไว้ที่ `document` (ทั้งหน้าเว็บ) แต่จะจางว่าจะทำงานเฉพาะเมื่อ element ที่ถูกคลิกมี class เป็น `btn-delete` ข้อดีคือใช้ได้แม้ปุ่มนั้นจะถูกสร้างขึ้นทีหลัง (เช่น ปุ่มที่มาจากผลการค้นหาด้วย AJAX) ต่างจากการผูก `.click()` ตรงๆ ที่จะได้แค่กับปุ่มที่มีอยู่ตอนหน้าเว็บโหลดเสร็จเท่านั้น

confirm('...') — คำสั่งของ browser ที่แจ้งเตือนข้อความถามยืนยัน ถ้าผู้ใช้กด "ตกลง" จะคืนค่า true โค้ดในวงเล็บ if จึงทำงานต่อ ถ้ากด "ยกเลิก" จะคืนค่า false แล้วจบการทำงานตรงนั้นทันที

\$(this).data('id') — ดึงค่าจาก attribute `data-id="..."` ที่ฝังไว้ในปุ่มตอน PHP สร้าง HTML ขึ้นมา (เช่น `data-id="<?= $row['id'] ?>"`) ทำให้รู้ว่ากำลังจะลบนักเรียนคนไหน

btn.closest('tr').remove(); — `closest('tr')` คือค้นหาจาก element ปุ่มที่กด ไล่ขึ้นไปหา parent element ที่ใกล้ที่สุดที่เป็น tag `<tr>` (แถวตารางที่ปุ่มนี้อยู่ใน) แล้ว `.remove()` คือลบ element นั้นออกจากหน้าเว็บทันที ทำให้แถวหายไปโดยไม่ต้องโหลดตารางใหม่ทั้งหมด

14. โหลดทีละบรรทัด: get_one.php / update.php + AJAX แก้ไข

```
<?php
require "connect.php";
$id = $_POST['id'];

$stmt = $conn->prepare("SELECT * FROM students WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();
$row = $stmt->get_result()->fetch_assoc();

echo json_encode($row);
?>
```

ไฟล์นี้ดึงข้อมูล "1 แถว" ของนักเรียนที่ระบุ id มาเก็บใน `$row` (เป็น associative array) แล้วใช้ `json_encode()` แปลง array นั้นให้เป็นข้อความรูปแบบ JSON (เช่น `{"id": "3", "name": "สมชาย", "age": "15", "class": "ม.2"}`) เพราะ JavaScript ฝั่ง browser อ่านข้อมูลแบบ JSON ได้ง่ายกว่า array ของ PHP โดยตรง (PHP กับ JavaScript เป็นคนละภาษา ต้องมีรูปแบบกลางที่ทั้งคู่เข้าใจตรงกัน ซึ่งก็คือ JSON)

```
<?php
require "connect.php";
$id = $_POST['id'];
$name = $_POST['name'];
$age = $_POST['age'];
$class = $_POST['class'];

$stmt = $conn->prepare("UPDATE students SET name=?, age=?, class=? WHERE id=?");
$stmt->bind_param("sisi", $name, $age, $class, $id);
$stmt->execute();

echo "success";
?>
```

bind_param("sisi", ...) — ตัวอักษร 4 ตัวต้องตรงกับลำดับและชนิดข้อมูลของตัวแปร 4 ตัวที่ตามมา: s=name (string), i=age (integer), s=class (string), i=id (integer) ลำดับต้องตรงกับลำดับเครื่องหมาย `?` ในคำสั่ง SQL ใดๆ ผิดลำดับแม้แค่ตัวเดียวจะทำให้บันทึกข้อมูลผิดพลาดแน่นอน

```
$(document).on('click', '.btn-edit', function(){
    let id = $(this).data('id');
    $.post('get_one.php', { id: id }, function(data){
        let student = JSON.parse(data);
        $('#edit_id').val(student.id);
        $('#edit_name').val(student.name);
        $('#edit_age').val(student.age);
        $('#edit_class').val(student.class);
        new bootstrap.Modal(document.getElementById('editModal')).show();
    });
});
```

JSON.parse(data) — ทำงานตรงข้ามกับ `json_encode()` ฝั่ง PHP คือแปลงข้อความ JSON ที่ได้รับกลับมาให้เป็น object ของ JavaScript ที่เรียกใช้ค่าข้างในได้สะดวก เช่น `student.name`

`$('#edit_name').val(student.name)` — นำค่าที่ได้ไปใส่ในช่อง input ของ modal (id="edit_name") ทำให้ผู้ใช้เห็นข้อมูลเดิมก่อนแก้ไข

`new bootstrap.Modal(...).show()` — คำสั่งของ Bootstrap ที่ใช้เปิดกล่อง Modal ขึ้นมาแสดงบนหน้าจอ

```
$('#btnSaveEdit').click(function(){
    $.post('update.php', {
        id: $('#edit_id').val(),
        name: $('#edit_name').val(),
        age: $('#edit_age').val(),
        class: $('#edit_class').val()
    }, function(response){
        if (response.trim() === 'success') {
            let id = $('#edit_id').val();
            $('#row-' + id + ' .col-name').text($('#edit_name').val());
            bootstrap.Modal.getInstance(document.getElementById('editModal')).hide();
        }
    });
});
```

เมื่อกดปุ่มบันทึกใน modal จะดึงค่าปัจจุบันจากทุกช่อง input ของ modal ส่งไปที่ update.php ผ่าน AJAX เมื่อได้คำตอบ "success" กลับมา จะใช้ `$('#row-' + id + ' .col-name').text(...)` เพื่ออัปเดตข้อความในตารางแถวที่ตรงกันให้ตรงกับค่าที่เพิ่งแก้ไข (โดยไม่ reload หน้า) แล้วปิด modal ด้วย `bootstrap.Modal.getInstance(...).hide()`

15. Error ที่เจอบ่อยตอนเริ่มต้น และวิธีแก้

ข้อความ Error	สาเหตุที่พบบ่อย	วิธีแก้
Connection failed / Access denied	ยังไม่ได้เปิด MySQL ใน XAMPP หรือใส่ username/password ผิด	เปิด XAMPP Control Panel แล้วกด Start ที่ MySQL ตรวจสอบว่า user/password ตรงกับที่ตั้งไว้
Unknown database 'xxx'	ยังไม่ได้สร้างฐานข้อมูลชื่อนี้ หรือพิมพ์ชื่อผิด	เข้า phpMyAdmin แล้วสร้างฐานข้อมูลให้ตรงชื่อกับใน connect.php
Undefined array key "xxx" (หรือ Notice: Undefined index)	เรียก \$_POST['xxx'] แต่ input ในฟอร์มไม่มี name="xxx" หรือฟอร์มยังไม่ถูก submit	ตรวจสอบว่า attribute name ในฟอร์มตรงกับ key ที่เรียกใน PHP ทุกตัวอักษร (พิมพ์เล็กใหญ่ต้องตรงกัน)
Call to a member function bind_param() on bool	คำสั่งใน prepare() เขียนผิด SQL syntax ทำให้ prepare() คืนค่า false แทนที่จะเป็น statement object	ตรวจสอบชื่อคอลัมน์/ตารางในคำสั่ง SQL ว่าสะกดถูกและมีอยู่จริงในฐานข้อมูล
หน้าเว็บว่างเปล่า ไม่แสดงอะไรเลย	PHP error แต่ปิดการแสดง error ไว้ (display_errors = Off)	เปิดไฟล์ php.ini ตั้ง display_errors = On แล้ว restart Apache หรือเพิ่ม <code>error_reporting(E_ALL);</code> <code>ini_set('display_errors', 1);</code> ไว้บนสุดของไฟล์ระหว่างทดสอบ
AJAX กดปุ่มแล้วไม่มีอะไรเกิดขึ้นเลย	ลืมใส่ไฟล์ jQuery หรือ URL ในปุ่ม \$.ajax เขียนชื่อไฟล์ผิด	เปิด Console ใน browser (กด F12) ดู error สีแดง ตรวจสอบว่าโหลด jquery.min.js สำเร็จหรือไม่ และชื่อไฟล์ปลายทางถูกต้อง
Modal ไม่เปิดขึ้นมา	ลืมใส่ไฟล์ bootstrap.bundle.min.js หรือ id ของ modal ไม่ตรงกับที่เขียนใน JavaScript	ตรวจสอบว่า id="editModal" ใน HTML ตรงกับที่ใช้ใน <code>document.getElementById('editModal')</code>

16. คำศัพท์ที่ควรรู้ (Glossary)

คำศัพท์	ความหมายแบบเข้าใจง่าย
Server-side	ส่วนที่ทำงานบนเซิร์ฟเวอร์ ผู้ใช้มองไม่เห็นโค้ด (เช่น PHP)
Client-side	ส่วนที่ทำงานบนเครื่องผู้ใช้ผ่าน browser (เช่น JavaScript, HTML, CSS)
Request / Response	การขอข้อมูล (จาก browser ไปเซิร์ฟเวอร์) และการตอบกลับ (จากเซิร์ฟเวอร์มา browser)
Superglobal	ตัวแปรพิเศษของ PHP ที่เรียกใช้ได้จากทุกที่ในโค้ด เช่น <code>\$_POST</code> , <code>\$_GET</code> , <code>\$_SERVER</code>
Query	คำสั่งที่ส่งไปสั่งงานฐานข้อมูล เช่น <code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code>
Prepared Statement	วิธีส่งคำสั่ง SQL ที่แยกค่าข้อมูลออกจากตัวคำสั่งอย่างปลอดภัย ป้องกัน SQL Injection
SQL Injection	การโจมตีโดยแทรกคำสั่ง SQL ปลอมผ่านช่องกรอกข้อมูล เพื่อขโมยหรือทำลายข้อมูล
AJAX	เทคนิคให้ browser คุยกับเซิร์ฟเวอร์เบื้องหลังได้โดยไม่ต้อง reload หน้าทั้งหน้า
jQuery	ไลบรารีของ JavaScript ที่ทำให้เขียนโค้ดจัดการหน้าเว็บและ AJAX ได้สั้นและง่ายขึ้น
JSON	รูปแบบข้อความมาตรฐานสำหรับส่งข้อมูลระหว่างภาษาโปรแกรมต่างชนิดกัน (เช่น PHP กับ JavaScript)
DOM	โครงสร้างของหน้าเว็บที่ JavaScript เข้าไปอ่าน/แก้ไของค์ประกอบต่างๆ ได้ (เช่น เปลี่ยนข้อความ, เพิ่ม/ลบ element)
Event	เหตุการณ์ที่เกิดขึ้นบนหน้าเว็บ เช่น click (คลิก), keyup (ปล่อยแป้นพิมพ์), submit (ส่งฟอร์ม)
Responsive	การออกแบบหน้าเว็บให้แสดงผลสวยงามเหมาะสมทั้งบนจอคอมพิวเตอร์และมือถือ

อ่านไฟล์นี้ควบคู่กับ "คู่มือสร้างเว็บเร่งด่วน.pdf" — เมื่อเข้าใจหลักการในเล่มนี้แล้ว โค้ดในเล่มนั้นจะไม่ใช่แค่โค้ดที่ท่องจำมา แต่เป็นโค้ดที่เข้าใจว่าทำไมต้องเขียนแบบนั้นจริงๆ